Laporan Tugas 4 Pemrograman Mobile



Disusun Oleh:

Vramudhia Egy Pratama 20240801382

Dosen Pengampu:

JEFRY SUNUPURWA ASRI , S.Kom., M.Kom.

Fakultas Ilmu Komputer

Program Studi Teknik Informatika

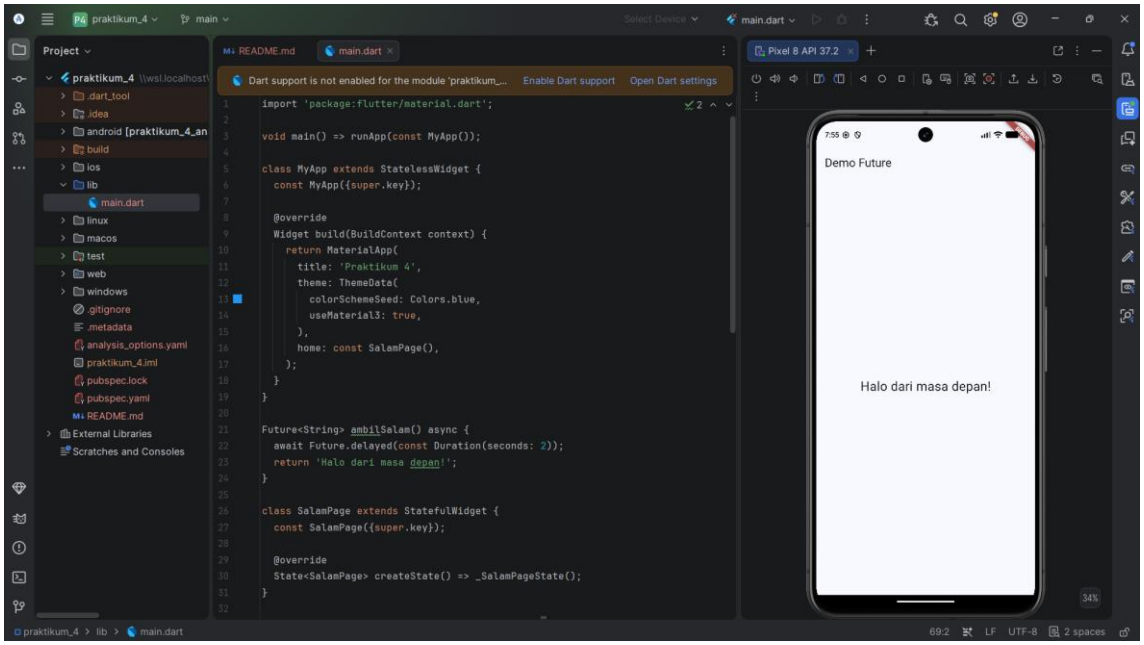


Foto 1 – Tampilan awal aplikasi dan penerapan Future

Pada foto pertama terlihat proses pembuatan aplikasi Flutter pada project **praktikum_4** menggunakan file main.dart. Pada bagian kode terdapat fungsi ambilSalam() yang menggunakan konsep **Future dan asynchronous programming**. Fungsi tersebut memberikan jeda selama 2 detik menggunakan Future.delayed() sebelum mengembalikan teks **“Halo dari masa depan!”**. Pada emulator terlihat halaman aplikasi dengan judul **“Demo Future”** dan teks “Halo dari masa depan!” di bagian tengah layar. Tampilan tersebut menunjukkan bahwa aplikasi berhasil dijalankan dan konsep asynchronous dapat digunakan untuk menjalankan proses yang membutuhkan waktu tanpa menghentikan keseluruhan aplikasi. Pada kode juga terlihat penggunaan MaterialApp, ThemeData, serta SalamPage sebagai halaman utama aplikasi.

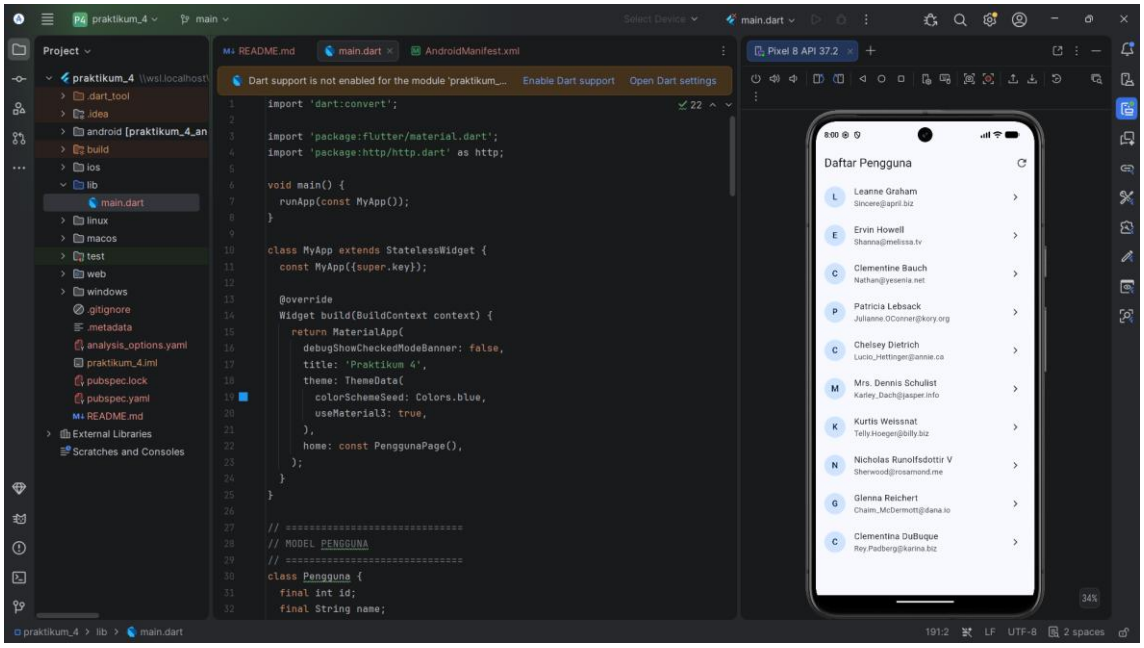


Foto 2 – Menampilkan daftar pengguna dari API

Pada foto kedua terlihat aplikasi menampilkan halaman **“Daftar Pengguna”** pada emulator. Halaman tersebut berisi beberapa data pengguna yang terdiri dari nama dan alamat email, seperti **Leanne Graham, Ervin Howell, Clementine Bauch, Patricia Lebsack**, dan pengguna lainnya. Setiap data ditampilkan dalam bentuk ListTile dengan avatar berupa huruf awal nama, kemudian terdapat ikon panah di sebelah kanan yang menunjukkan bahwa data tersebut dapat dipilih untuk melihat informasi lebih detail. Pada bagian kode terlihat penggunaan package http dan dart:convert, yang digunakan untuk melakukan komunikasi dengan API dan mengubah data JSON menjadi objek Dart. Dengan demikian, foto ini menunjukkan implementasi pengambilan data dari API kemudian menampilkannya secara dinamis dalam bentuk daftar pada aplikasi Flutter.

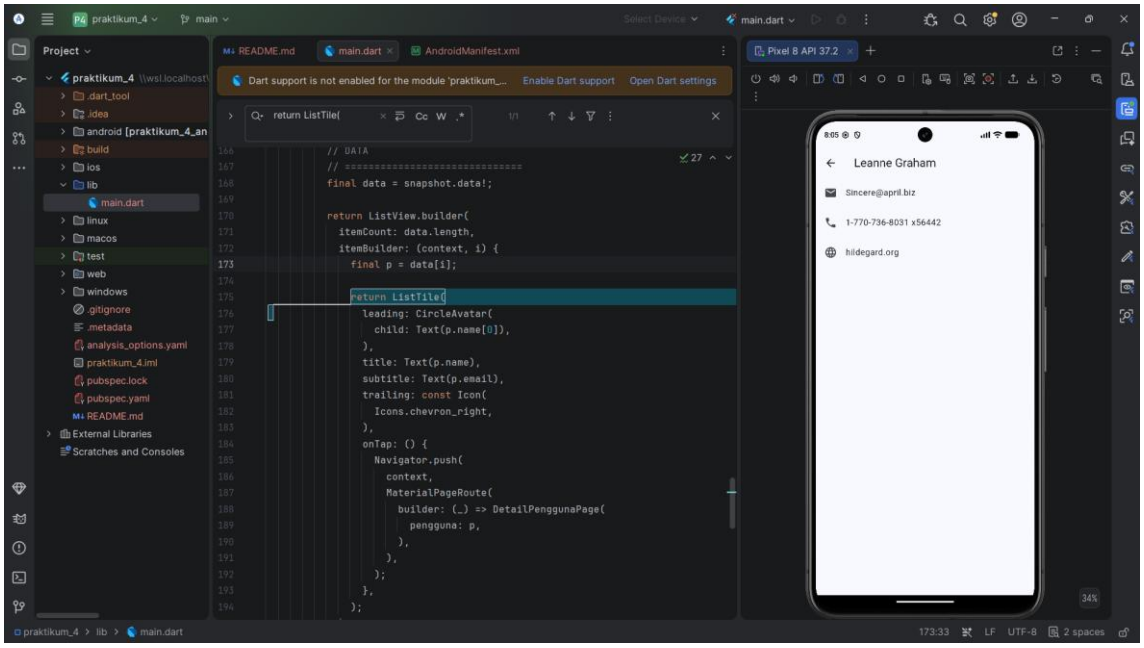


Foto 3 – Menampilkan detail pengguna

Pada foto ketiga terlihat halaman detail dari salah satu pengguna, yaitu **Leanne Graham**. Ketika pengguna pada daftar sebelumnya dipilih, aplikasi berpindah ke halaman detail yang menampilkan beberapa informasi seperti **email, nomor telepon, dan website**. Perpindahan halaman dilakukan melalui Navigator.push() dan MaterialPageRoute, kemudian data pengguna dikirimkan ke halaman DetailPenggunaPage. Hal ini menunjukkan penerapan navigasi antarhalaman pada Flutter. Pada kode juga terlihat bahwa data yang telah diperoleh dari API digunakan sebagai objek Pengguna, sehingga halaman detail tidak perlu mengambil data ulang secara manual, tetapi cukup menggunakan data pengguna yang dipilih dari daftar.

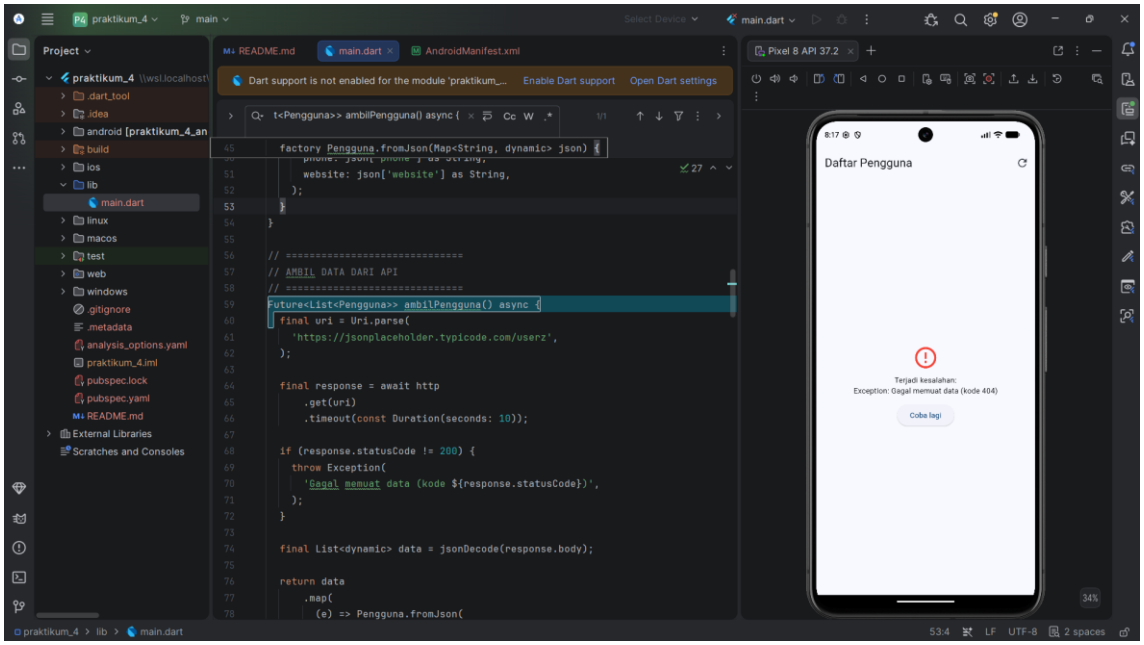


Foto 4 – Percobaan mengambil data API dengan endpoint yang salah

Pada foto keempat terlihat aplikasi mengalami error ketika melakukan proses pengambilan data dari API. Pada kode terlihat URL yang digunakan adalah endpoint **https://jsonplaceholder.typicode.com/user**, sedangkan aplikasi menampilkan pesan **"Exception: Gagal memuat data (kode 404)"**. Kode 404 menunjukkan bahwa resource atau endpoint yang diminta tidak ditemukan. Program juga memiliki pengecekan response.statusCode != 200, sehingga apabila server memberikan status selain 200, aplikasi akan menghasilkan exception dan menampilkan pesan kesalahan kepada pengguna. Foto ini menunjukkan bahwa aplikasi sudah memiliki mekanisme **error handling** untuk menangani kondisi ketika data dari API gagal diperoleh.

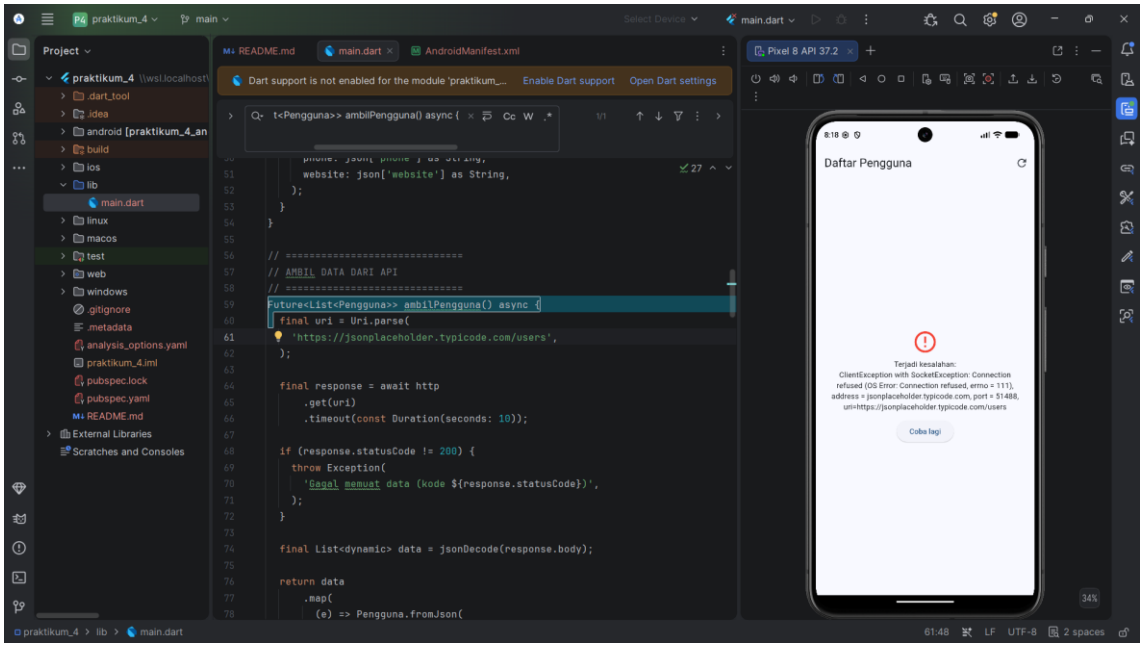


Foto 5 – Perbaikan endpoint tetapi terjadi masalah koneksi

Pada foto kelima terlihat endpoint pada kode sudah menggunakan URL **https://jsonplaceholder.typicode.com/users**, yaitu endpoint yang sesuai untuk mengambil daftar pengguna. Namun, aplikasi masih gagal mendapatkan data dan menampilkan pesan **"ClientException with SocketException: Connection refused"**. Berbeda dengan foto sebelumnya yang menghasilkan status HTTP 404, masalah pada foto ini terjadi pada tingkat koneksi sehingga aplikasi tidak berhasil terhubung ke server API. Kode tetap menggunakan http.get() dengan batas waktu timeout selama 10 detik. Dengan adanya timeout tersebut, aplikasi tidak akan menunggu koneksi tanpa batas waktu. Foto ini menunjukkan pentingnya penanganan error jaringan dalam aplikasi yang mengambil data dari internet.

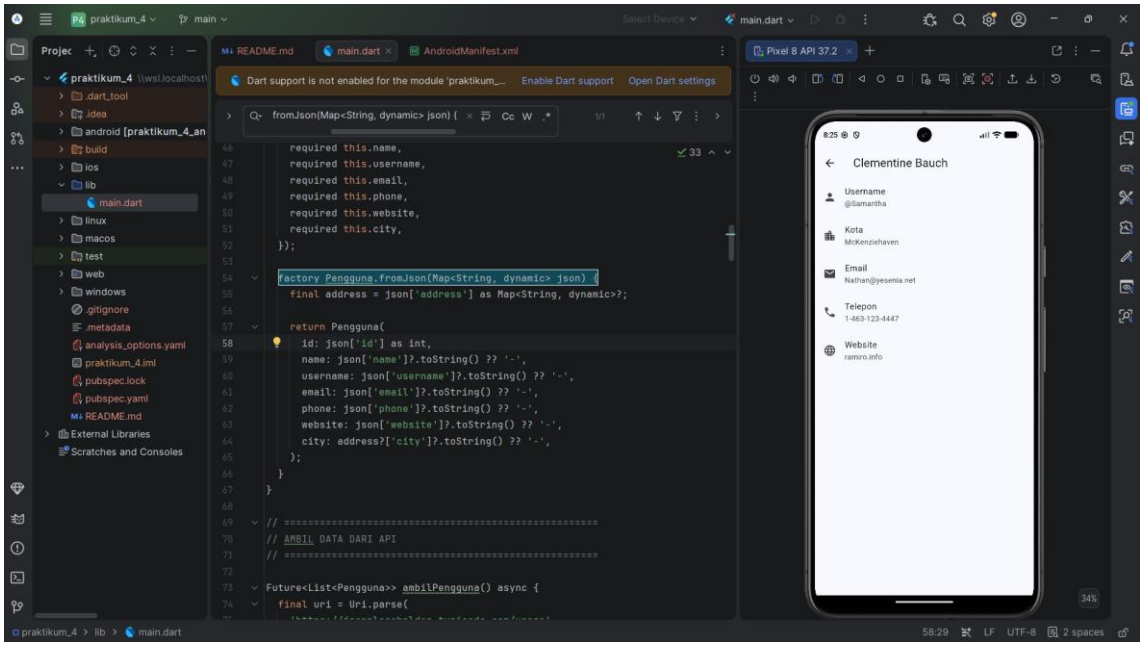


Foto 6 – Konversi data JSON menjadi objek Pengguna

Pada foto keenam terlihat kode factory Pengguna.fromJson(Map<String, dynamic> json) yang digunakan untuk mengubah data JSON dari API menjadi objek Pengguna. Data seperti id, name, username, email, phone, website, dan city diambil dari struktur JSON menggunakan key masing-masing. Pada emulator terlihat halaman detail pengguna **Clementine Bauch** yang menampilkan informasi username, kota, email, telepon, dan website. Hal ini menunjukkan bahwa proses parsing JSON berhasil digunakan untuk mengubah data mentah dari API menjadi data yang lebih mudah digunakan oleh aplikasi. Dengan menggunakan model Pengguna, struktur program menjadi lebih terorganisasi karena data API memiliki representasi objek yang jelas di dalam aplikasi.